CampusOne

One record per student: attendance, internal marks and fees for a multi-campus college ERP

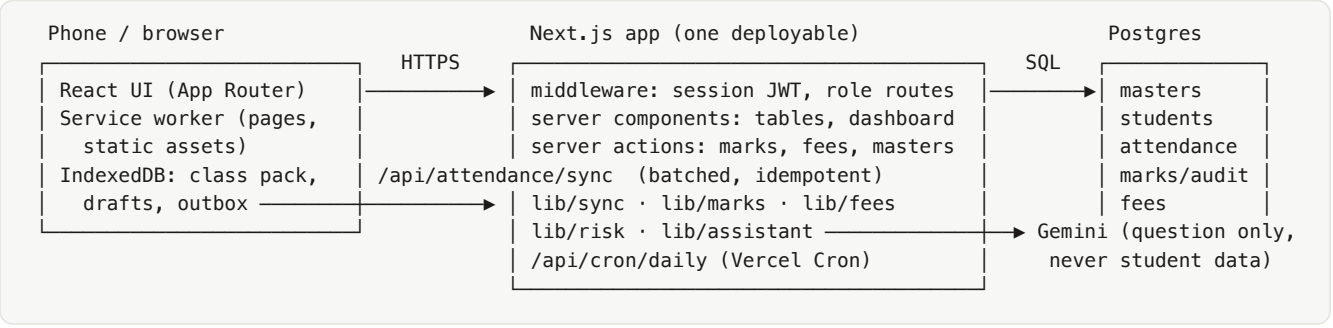
GitHub	https://github.com/ambitiouswithayush/campusone
Live app	Not deployed at the time of writing. The repository is ready for Vercel + managed Postgres (see deployment.md). The full demo runs locally in about two minutes with no database install.
Demo logins	Faculty `priya.das@campusone.test` · Office admin `admin@campusone.test` · password `Demo@2026`
Stack	Next.js 15 · TypeScript · Postgres (Drizzle ORM; PGlite locally) · Tailwind CSS 4 · Service worker + IndexedDB · Vitest

1. What was built

This is the **core build slice**, working end to end for **one campus** (the Engineering College) with two roles, **faculty** and **office admin**. It runs on realistic seed data built around the three sample students in the brief.

BRIEF REQUIREMENT	WHAT WORKS
Masters for one programme	Campuses, departments, programmes, batches (intake + section), subjects with the 40-mark internal scheme, faculty, teaching assignments, weekly timetable and academic calendar. Subjects, faculty, teacher reassignment and calendar entries are editable; every change is audited.
Mobile attendance, offline in class	A class pack (rosters and timetable) is cached on the phone. Every student starts as present and the teacher taps only the absentees. Every tap is saved to the phone; submissions queue in an outbox and sync when signal returns. Retries are idempotent, conflicts are resolved by rule, and the server enforces a 48-hour edit window.
Alerts below 75%	After each sync and nightly, an automated shortage alert goes to the parent (a mocked SMS, in English or Odia). It is de-duplicated to one per student per week and held back when parental consent is missing.
Internal marks with moderation lock	A grid with inline validation and keyboard entry. The workflow is draft → submitted (moderation) → locked; unlocking needs a written reason. Every change is logged, and locked sheets export in the university upload format.
Fee dues	Fee structures by programme and category, scholarships, dues ledger, bank-counter receipts, test-mode payment links with an idempotent callback, printable receipts, dues CSV and Tally XML export.
AI: at-risk flag	An explainable score from attendance and marks levels and trends. It creates a *mentor task* only, refuses to judge when there is too little data, and treats a mentor's dismissal as feedback.
AI: plain-English questions (optional)	The question becomes a whitelisted JSON filter spec, which is validated, scoped to the user's campus and run read-only. The answer shows the filters, the row count and which engine answered. A rule parser takes over when the model is unavailable.
Dashboards	The campus dashboard shows the share of classes marked the same day (the brief's 95% target), shortage and risk counts, weekly attendance, a list of classes still to chase, marks-lock progress and automation activity.

2. Architecture and why



- **One Next.js monolith, not microservices.** Phase 1 is ₹9 lakh and 8 weeks. One codebase, one deploy and one set of types from the database to the screen keeps it buildable and maintainable by a small team. Domain logic lives in plain `src/lib/*` modules that take a database handle, so it is unit-testable and could later move behind a separate API without rewriting.
- **Postgres.** The domain is relational: students, batches, sessions, marks, invoices. It needs transactions (sync conflicts, payment callbacks), row locks (`SELECT … FOR UPDATE`) and aggregate reporting for university, AICTE and NAAC. Any managed Postgres works.
- **PGlite locally and in tests.** The same Postgres engine compiled to WebAssembly. Reviewers run the app with no database install, and the 38 tests run against real Postgres semantics (unique indexes, row locks, sequences) rather than mocks. Production uses `DATABASE_URL` with node-postgres, through the same Drizzle schema and migrations.
- **Offline through the platform, not a sync framework.** A small service worker caches pages and assets. IndexedDB holds the class pack, in-progress drafts and an outbox. The server is the source of truth and decides conflicts in one function (`lib/sync.ts`). That made the rules simple to explain to the client and to test. A CRDT or a replicated local database would be more machinery than "mark 60 students and send" needs.
- **Security in layers.** Middleware blocks routes by role. Every server action and API call re-checks the role, and every query is scoped by the campus taken from the session, never from the request. Teacher ownership is checked per class.

3. Data model

AREA	TABLES	NOTES
Masters	campuses, departments, programmes, batches, subjects, faculty, teaching_assignments, timetable_slots, calendar_events	A *batch* is programme + intake year + section. A *teaching assignment* (batch × subject → teacher) is the anchor for attendance and marks, so reassigning a teacher moves both.
People	users, students	Users carry a role and campus (plus faculty_id for teachers). Students hold `parent_consent_at` for the DPDP Act and the guardian's message language.
Attendance	attendance_sessions (UUID PK), attendance_records	The session ID is generated on the phone, which makes retries safe. Unique (batch, date, period) means a class can only be marked once. `marked_at` and `synced_at` are stored separately; present and absent counts are denormalised for fast lists.
Marks	assessment_components, marks, mark_sheets	Components per subject (Quiz 1 out of 5, Mid-sem 20, Assignment 10, Quiz 2 5). Each subject-and-batch sheet carries its workflow state.

Fees	fee_structures, scholarships, fee_invoices, payments, receipt_seq	The balance is computed (gross – concession – successful payments), never stored. A unique `reference` stops a replayed callback from crediting twice. Receipt numbers come from a Postgres sequence.
AI and audit	risk_flags, messages, audit_log, assistant_queries	A risk flag *is* the mentor task. Messages carry a unique dedupe key. The audit log stores before and after values plus a reason for marks, attendance corrections, payments and masters. Every assistant question and the spec it ran is logged.

4. Main flows

4.1 Attendance in class: the hard part

```
Staff room (Wi-Fi)        Classroom (no signal)                      Back in signal
GET /api/faculty/pack  →  open class → all present by default       outbox.flush() on 'online' event,
 rosters + timetable       tap absentees (each tap → IndexedDB draft)   every 20 s while pending, on focus
 + existing marks          Submit → outbox item {uuid, markedAt, …}   POST /api/attendance/sync (≤ 50)
 + 14 days of holidays     list shows "Saved on phone"                   ↳ per-session result:
```

SERVER SEES	RESULT	WHY
Same UUID, same marks	duplicate	The request timed out on weak Wi-Fi and the phone resent it. This is a harmless no-op.
Same UUID, different marks	updated	The teacher corrected the class. The before and after values go to the audit log.
Other UUID, later `markedAt`	replaced	The later decision wins. The older copy is kept in the audit log.
Other UUID, earlier `markedAt`	conflict	The phone shows both copies; the teacher chooses *Keep server copy* or *Use mine* (an override).
Wrong teacher, holiday, future date, not on the timetable, window closed	rejected	Shown on the phone with the reason. The office can still correct the record after the window.

Edge cases handled: the 48-hour window is measured from **when the class was marked**, so a phone that only finds signal two days later still counts. A phone clock running ahead is clamped to server time with a warning. Students who join after the pack was downloaded are reported, not silently dropped. If the phone stays offline past midnight, it rebuilds the day's classes from the cached timetable. Sign-out wipes cached rosters (staff phones are shared), but first warns if unsynced classes would be lost. **Target: under 30 seconds per class.** One tap per absentee plus one submit. The phone shows the elapsed time ("Saved in 14 s · 2 absent").

4.2 Internal marks

```
draft ──(faculty submits; all cells filled or AB)──▶ submitted ──(exam cell locks)──▶ locked ──▶ university CSV
  ▲                                                    │  office moderates; edits logged     │
  └──────────────(return, with reason)─────────────────┘◀──────(unlock, reason ≥ 10 chars)─────┘
```

Faculty edit only drafts and the office edits only during moderation. Nobody edits a locked sheet. Validation runs on the client (inline red cells) and again on the server (component maximums, half-mark steps, AB without a score, student must belong to the batch).

4.3 Fees

Each invoice is generated from the structure for the student's programme and category, less scholarships. Counter payments need a unique bank reference and cannot exceed the balance. A payment link creates a pending payment. The mock gateway callback locks the row, re-checks the balance (another payment may have cleared it) and issues a receipt exactly once; replays return the same receipt. Tally export writes Receipt vouchers as Tally Prime XML for a date range.

5. How the AI works and what happens when it is wrong

5.1 At-risk flag

Inputs: overall attendance, its change over the last 3 weeks against earlier weeks, the internal-marks average, the latest assessment against earlier ones, and the number of subjects under 40%. **Output:** a score from 0 to 100 with a plain sentence for every point added. 50 or more is "high", 30–49 is "watch".

Why a transparent points model instead of a trained classifier: the trust has no historical labels yet (who actually failed or dropped out), so any "trained" model would be fitted to nothing. Mentors must be able to see *why* a student is flagged. The weights are an ASSUMPTION to recalibrate after one semester of outcomes, using the dismissal notes collected below. Seeded example: Ankit Panda scores 58 (high) because of attendance of 68.2%, a 30-point recent drop and a falling latest assessment. Subhashree Mohanty (81.5%, 33/40) is not flagged.

FAILURE MODE	HANDLING
Too little data	With fewer than 20 classes and no marks, the model does not judge and reports "Not enough data". With one signal it says "Based on attendance only".
False positive	The mentor clicks *Not at risk* with a note. It is not re-raised for 14 days unless the score rises by 15 or more. Dismissals are stored for recalibration.
The student recovers	An untouched open task closes automatically with a note on the next scan.
Harm	A flag only creates a mentor task. It never changes marks, attendance or exam eligibility. Attendance and marks rules are fixed settings in `lib/config.ts`, not AI.

5.2 Plain-English questions

```
"How many second-year students are below 75% this month?"
   │ Gemini (temperature 0, JSON mode) gets the question + programme list — never student data
   ▼
{ output:"count", filters:{ yearOfStudy:2, attendance:{op:"lt", value:75, period:"this_month"} } }
   │ zod: whitelist of 9 filters, bounded values, unknown keys stripped, unknown programmes dropped
   ▼ compiled to parameterised Drizzle SELECTs + campus scope from the session  (no SQL from the model)
Answer: N students · chips "Year 2" "Attendance below 75% this month" "Campus scope" · engine: rule parser
```

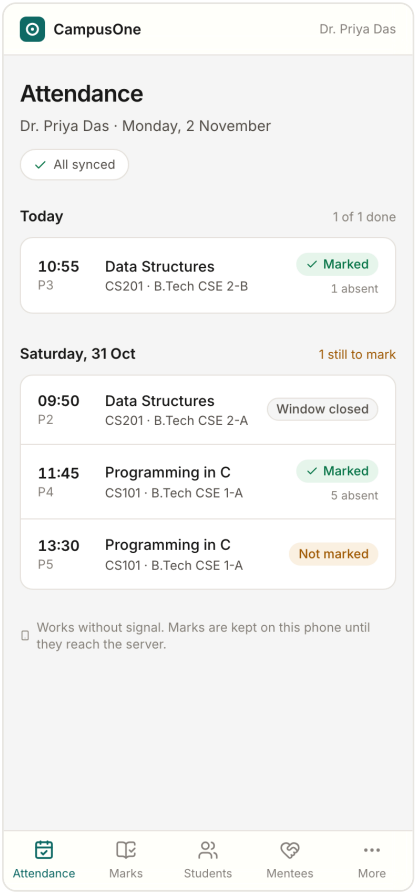
WHEN IT IS WRONG OR UNSURE	HANDLING
The model misreads the question	The filters are shown as chips next to the row count, so staff can check what actually ran. Every question and spec is logged.
The model is down, slow (10 s) or returns invalid JSON	The deterministic rule parser answers instead, and the answer says which engine was used.
Nothing maps to a filter	The assistant answers "I couldn't turn that into a report…" with examples. It does not guess.
Part of the question cannot be expressed	The model fills `unsupported`, which is shown as a warning above the result.
Write requests ("delete…", "send…")	Refused before the model is called. The executor cannot write anyway, because it only issues SELECTs.
Data the user may not see	Campus and role scope are applied in code; the spec cannot widen them. A test proves an Engineering admin asking for the MBA programme gets 0 rows.

A thin period such as "this month" on the 2nd	A warning says roughly how many classes fall in the period and suggests a wider one.

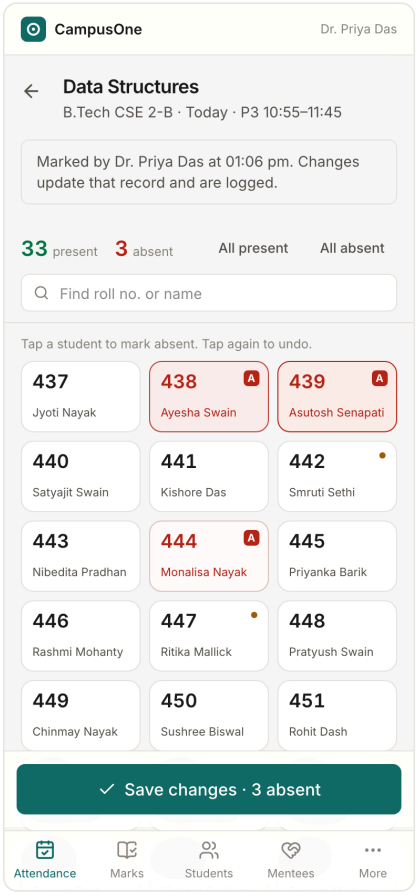
6. UI and design decisions

The client asked for calm, minimal and consistent screens: "nothing on screen that lacks a purpose". The decisions:

- **One layout pattern everywhere:** a page header (title, one line of context, at most two actions), then cards holding a table or list. A sidebar on desktop becomes a bottom tab bar on phones. Faculty see 4 menu items and the office sees 7.
- **A restrained palette:** warm neutrals, one teal accent for actions and selection, and three status colours that only ever mean something: red for shortage or overdue, amber for watch or awaiting, green for done or locked. Red never decorates.
- **Numbers first:** tabular figures, Indian digit grouping (₹30,04,000), attendance bars with a tick at 75%, so a shortage is visible before it is read.
- **Every state designed:** skeletons while loading, an error card with retry if a page fails (the menu stays usable), empty states that say what to do next, inline errors at the field, toasts for confirmations, and offline banners that explain what still works.



Faculty home: today's classes and the last 2 days still open for correction, with sync state.

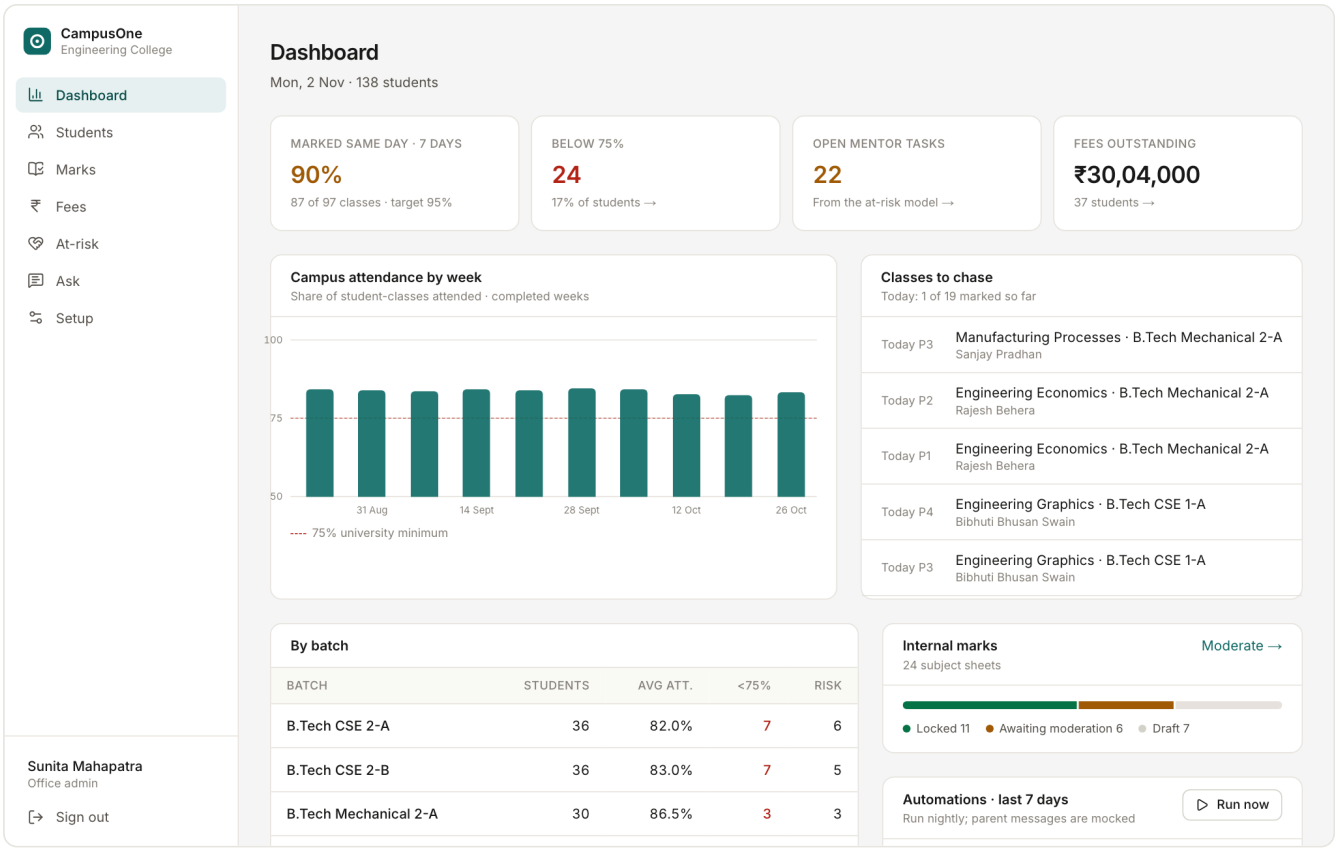


Taking attendance: all present by default; big roll-number tiles; amber dot = already below 75%.

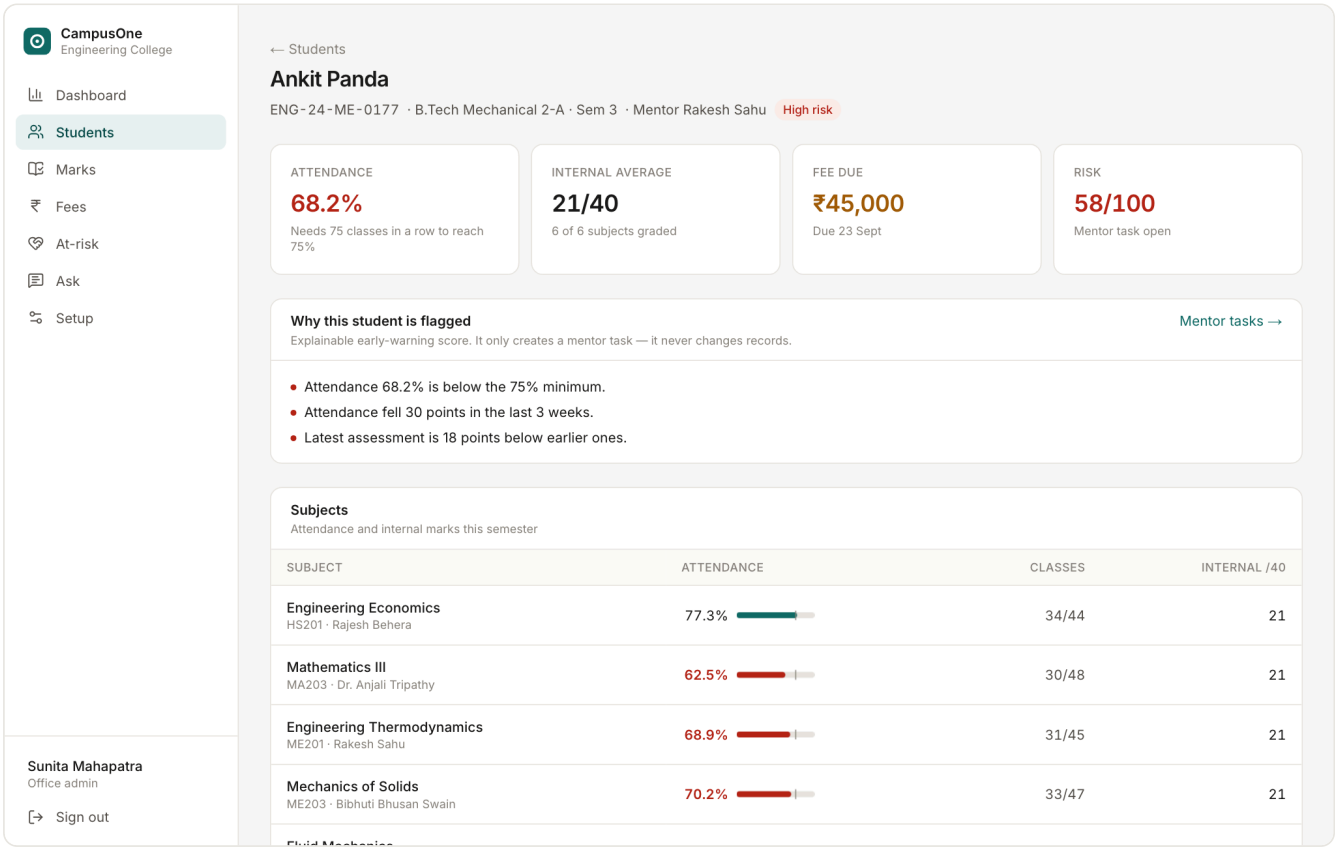
Why tiles with the last three roll digits: teachers call roll numbers, not names. Three columns fit a 360 px phone without scrolling sideways, and the tap target is about 110×68 px, so it is easy to hit while standing.

Why "all present" by default: most classes have 0–5 absentees, so the work is proportional to absences rather than class size.

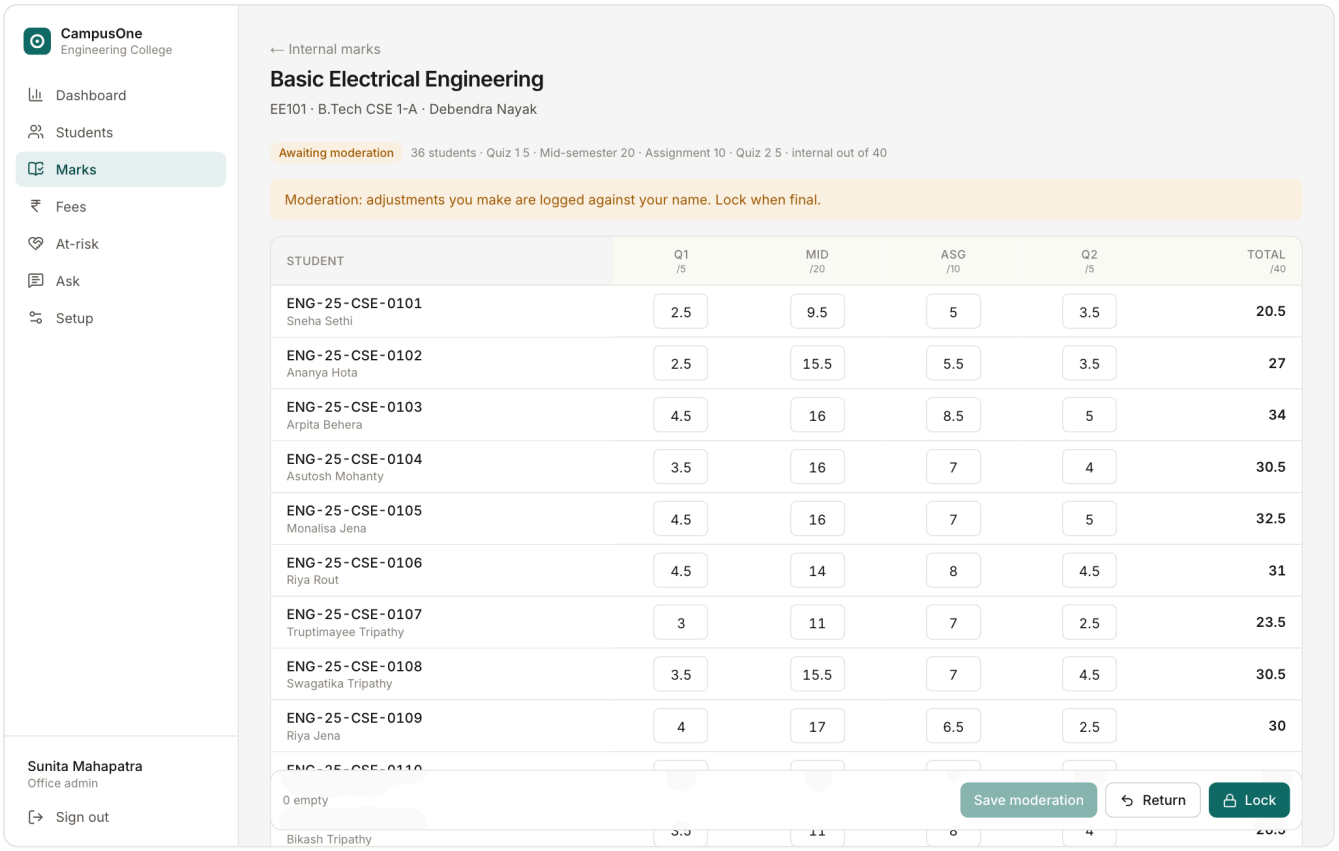
Why the sync chip is always visible: trust. The teacher should never wonder whether attendance "went through".



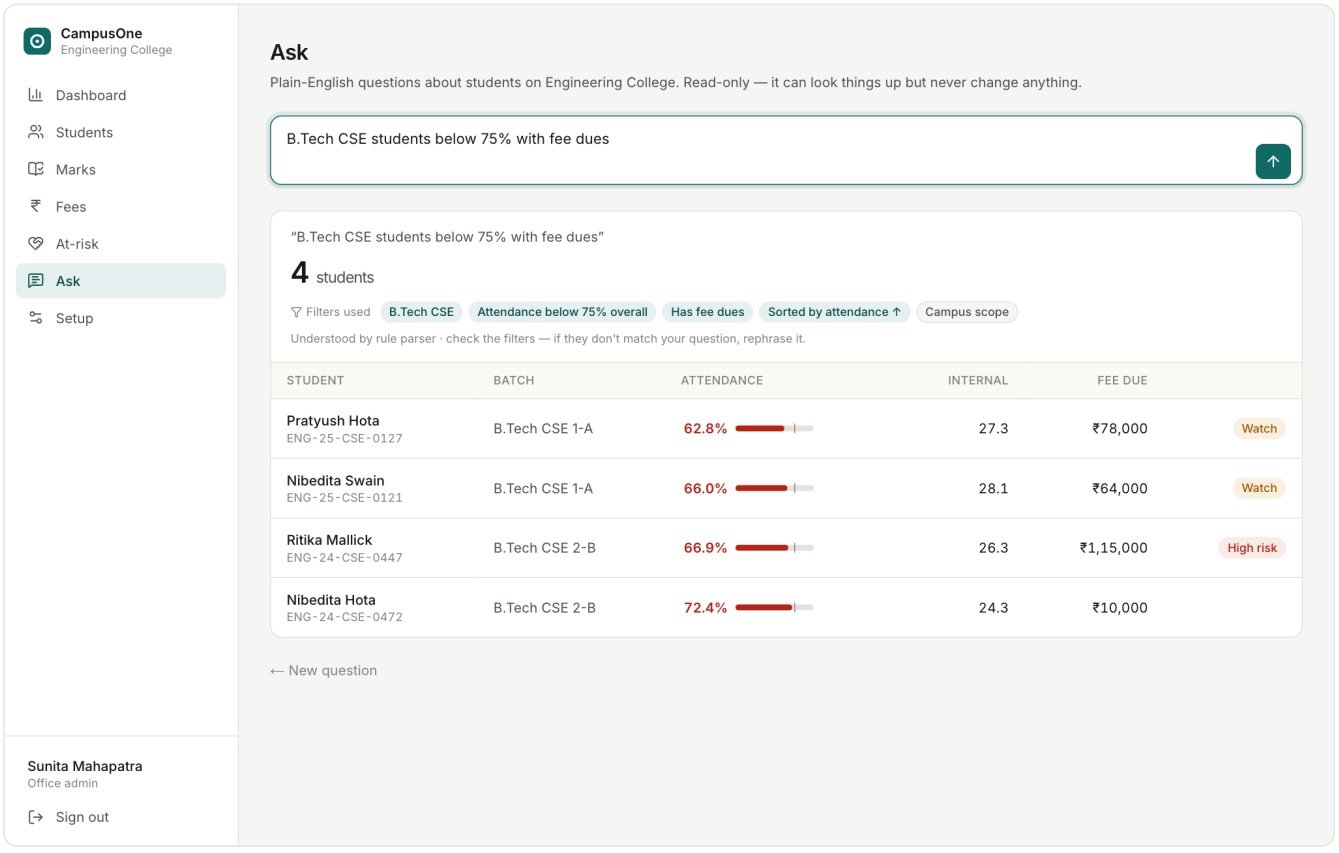
Campus dashboard: the same-day marking rate leads because it is the client's success metric.



One student record: attendance, internals, fees, risk reasons and parent messages in one place.



Mark sheet under moderation: spreadsheet-like entry, Enter moves down, invalid cells turn red.



Ask: the answer leads with the row count and the filters actually used.

7. Security, privacy and compliance

- **Authentication:** bcrypt password hashes and an HS256-signed, httpOnly, SameSite=Lax session cookie (12 hours). Login is rate-limited per email and compares in constant time for unknown emails. Redirects after login are restricted to same-site paths.
- **Access control:** role checks in middleware and again in every action and route. Campus scope comes from the session. Faculty reach only the classes they teach, plus their mentees. Faculty cannot see fees (data minimisation). Records outside scope return 404, not 403, so their existence is not revealed.
- **Input validation:** zod on every API body and server action; whitelisted assistant specs; parameterised SQL only.
- **DPDP Act 2023:** consent is recorded per student; minors without parental consent are flagged in their profile and get no parent messages. Only the question text, never student data, is sent to the AI model. Payment pages show a first name and a masked roll number.
- **Exam audit:** every mark change, submission, lock, unlock (with reason) and export is logged with the actor and time.
- **Secrets:** none in the repository. `.env*` files are ignored, `npm run setup` generates a local secret, and the cron endpoint uses a constant-time bearer check.

8. Assumptions

Everything below goes beyond what the problem statement says. Each is a setting or seed value that can be changed.

1. **A1** Seven 50-minute periods, Monday to Saturday (bell times in `lib/config.ts`).
2. **A2** The internal scheme for every subject is Quiz 1 (5) + Mid-sem (20) + Assignment (10) + Quiz 2 (5) = 40. Partial schemes are scaled to 40.
3. **A3** The sample's single "Internal Marks of 40" is the student's average across subjects. The 75% rule applies to overall attendance; per-subject figures are shown alongside.
4. **A4** University upload format: CSV with `RegdNo, SubjectCode, InternalMarks, MaxMarks, Status(P/AB)`. The real university template was not provided, and the exporter is one function to adapt.
5. **A5** Faculty may correct attendance for 48 hours after a class ends; after that only the office can.
6. **A6** Roll numbers follow the sample (`ENG-24-CSE-0412`). To reproduce the sample percentages exactly with whole-number class counts, the seed has the sample students report 0–5 classes late in week 1.
7. **A7** Fee amounts (₹1,40,000 B.Tech General/OBC/EWS; ₹1,25,000 SC/ST without the development fee; ₹2,10,000 MBA) and scholarships (Post-Matric ₹90,000, Merit ₹25,000) are illustrative. Tally ledger names must be mapped to the trust's chart of accounts.
8. **A8** Parental consent is captured on the admission form for every student; messages go only where consent is recorded. The Odia message templates need review by a native speaker before go-live.
9. **A9** "Office admin" covers the exam cell and accounts in this phase; finer roles come later. The timetable is a master (generating it is out of scope).
10. **A10** At-risk weights are hand-set (see 5.1). Gemini is the default model because it has a free tier for a pilot; the adapter is about 50 lines and can point at any provider.
11. **A11** The core build asks for one campus and masters for one programme, but the sample data spans three programmes on two campuses (B.Tech CSE, B.Tech Mechanical, MBA). The working slice is the Engineering College with its two B.Tech programmes, so both engineering sample students sit in it. Farheen Ali (MBA) is seeded on a second, Management campus with its own office admin; it is not built out further, but it shows campus isolation. A third campus (Pharmacy) exists as a master record only.
12. **A12** "Standard reports run automatically" is read as: the nightly job sends shortage alerts and fee reminders and runs the risk scan; reports (marks CSV, dues CSV, Tally XML) are generated on demand. Scheduled university, AICTE and NAAC reports are cut (section 10).

9. Testing

38 automated tests (`npm test` , about 7 s) run on a fresh in-memory Postgres seeded with demo data:

- **Sync:** idempotent retry, correction, conflict, newer copy replaces, override, wrong teacher, holiday, future date, not on the timetable, the edit window judged on `markedAt` , office correction after the window, clock-skew clamp, foreign students ignored.
- **Marks:** totals and scaling, validation, the role and state edit matrix, no submission with empty cells, lock, unlock reason required, the audit trail of every step, the university CSV format.
- **Risk:** a healthy student is not flagged, Ankit's profile scores high with reasons, insufficient-data and single-signal cases, no duplicate tasks on rescan, dismissal suppression.
- **Assistant:** the brief's example question, combined filters, unknown questions, whitelist validation, campus isolation (including a question about another campus's programme returning 0 rows rather than everyone), write refusal, and the three sample students matching the brief exactly.
- **Fees:** balance and concession bounds, overpayment and duplicate-reference rejection, callback idempotency with one audit entry per gateway outcome, double payment through two links, Tally export scoped to the campus.

The tests caught a real bug: the rule parser read "20,000" as 20. A final review against the brief caught two more, both now fixed and covered by tests: asking an Engineering admin's assistant about MBA students returned every Engineering student instead of none, and mock-gateway payments were not written to the audit log. The suite seeds data relative to today, so it was run with the clock set to each of the next 21 days (Sundays and holidays included) to make sure it passes whenever a reviewer runs it. Beyond the suite, a production build was checked in a headless browser on a 390 px phone: a reload with the network cut still loads the class list, a class marked offline syncs on reconnect, and faculty get 403 from admin APIs.

10. What I cut, and what comes next

Cut from this phase (on purpose)

- Admissions: applicant import, document checklist, roll-number allocation. The `students` table is ready for it.
- Student and parent logins (parents get mocked SMS only), staff leave, the trust-level dashboard across campuses.
- Real SMS or WhatsApp and payment gateways. Both are mocked behind small functions.
- University, AICTE and NAAC report formats beyond the internal-marks CSV, and scheduling standard reports (they are generated on demand today).
- Bank-statement reconciliation (the brief's "10 days to 1"). Unique references and Tally export prepare for it.
- Odia interface text (only parent messages are in Odia, as the brief asks).
- Distributed rate limiting, a Content Security Policy, and a password reset flow.

Next (in priority order)

1. **Pilot on one department for 2 weeks** and measure seconds per class and the same-day rate against the 95% target.
2. Admissions import with a document checklist and DPDP consent capture, so a student exists once from day one.
3. Bank statement CSV matching against references for one-day reconciliation.
4. Results-day readiness for 7,800 concurrent students: static, per-student results pages behind a CDN, a read replica, load testing at 3× peak. The core is stateless, so it scales horizontally.
5. Recalibrate the risk weights with the first semester's outcomes and dismissal notes; consider logistic regression once labels exist.
6. Expand to the management and pharmacy campuses (the data model and campus scoping already support it; the demo already seeds the Management campus with MBA students and a Pharmacy campus record), then the trust dashboard and NAAC reports.

Honest limits: the session secret rotates only by redeploy; the rate limiter is per instance; on a teacher's own phone the cached roster stays until sign-out or the next download; receipt numbers come from a sequence, so a rolled-back payment can skip a number (the audit log explains it).